VL 03 · Das Web und seine Protokolle

Was zwischen URL und Seite passiert

Prof. Dr. Pascal Laube · HTWG Konstanz · WS 2026/27

Ihr tippt eine Adresse ein …

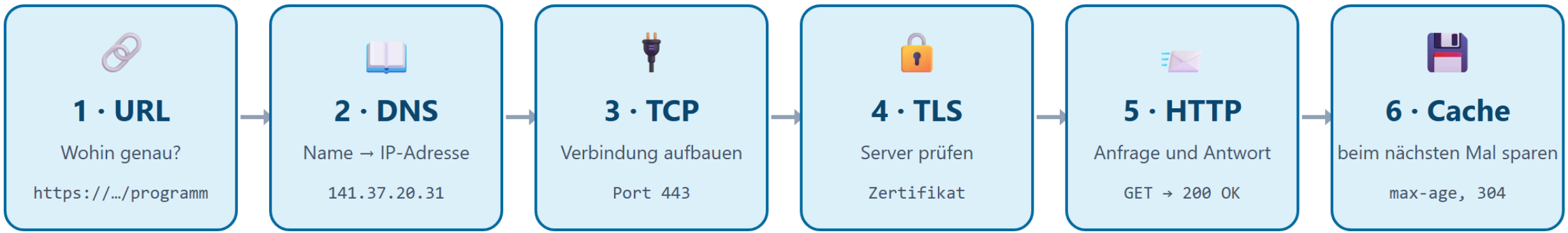
… und einen Augenblick später steht die Hackathon-Seite da.

> **Frage an euch:** Was muss in diesem Augenblick alles passieren? Sammelt in zwei Minuten mit eurem Nachbarn möglichst viele Schritte.

Warum sich das lohnt, obwohl am Ende ein Framework die Arbeit macht: Diese Protokolle sind seit Jahrzehnten stabil. React, Vue oder das nächste Framework ändern nichts daran, dass jede Seite und jede API-Anfrage genau diesen Weg nimmt. Wer ihn kennt, findet Fehler schneller.

Eine Anfrage, sechs Stationen

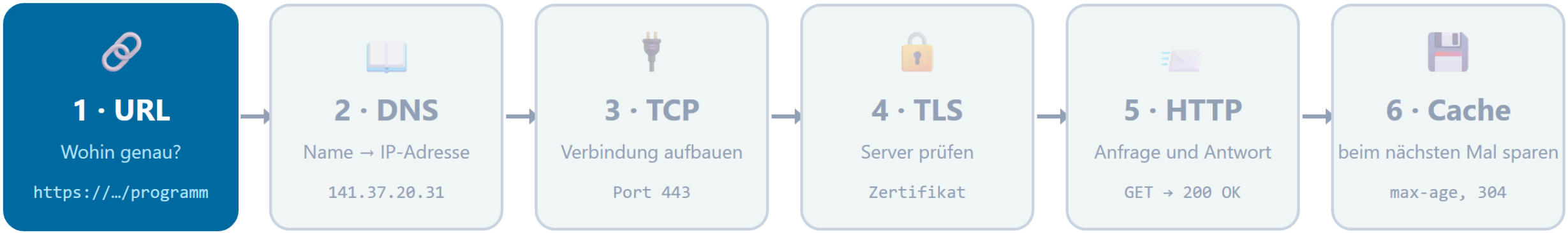
Was passiert, wenn ihr die Hackathon-Seite aufruft



Danach baut der Browser die Seite auf: das ist Thema von VL 6.

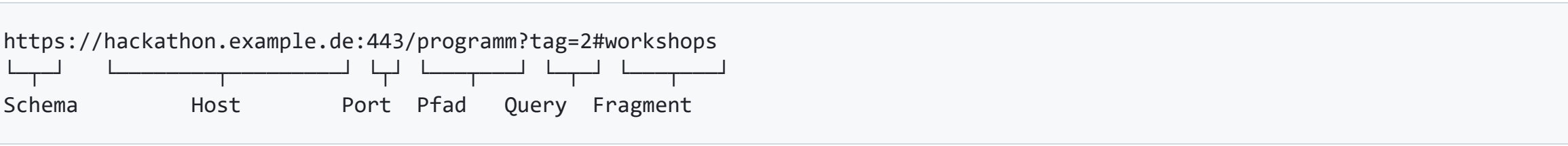
Eine Anfrage, sechs Stationen

Was passiert, wenn ihr die Hackathon-Seite aufruft



Danach baut der Browser die Seite auf: das ist Thema von VL 6.

Die URL: wohin genau?



🌐 Host

Der Name des Servers. Den löst gleich DNS auf.

📁 Pfad und Query

Welche Ressource, mit welchen Parametern. Beides kommt beim Server an.

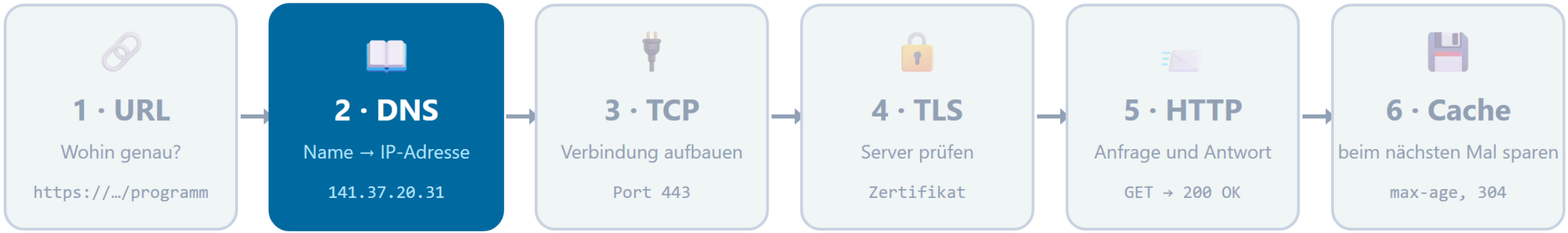
🚀 Fragment

Nach dem `#`. Bleibt im Browser und wird **nie** an den Server geschickt.

Für M1: Gute URLs beschreiben die Ressource, nicht die Technik. `/events/42/anmeldung` statt `/page.php?id=42&action=3`.

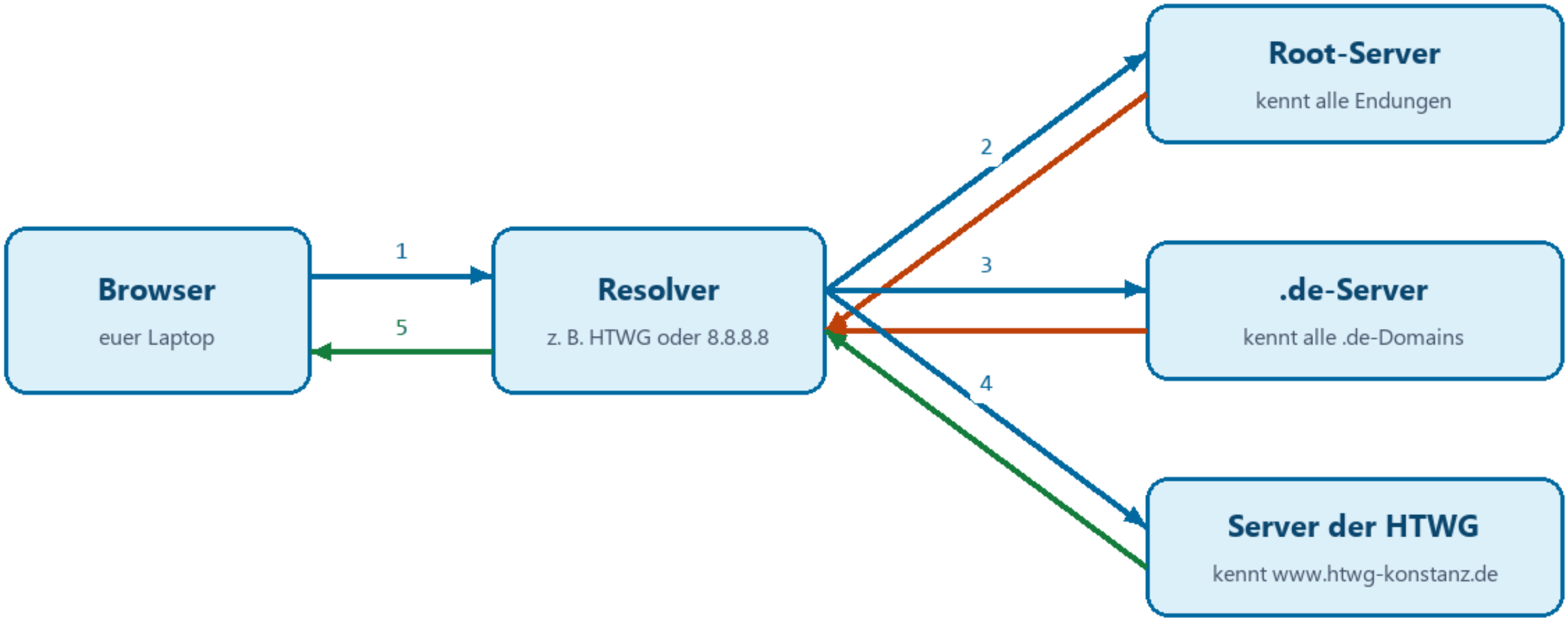
Eine Anfrage, sechs Stationen

Was passiert, wenn ihr die Hackathon-Seite aufruft



Danach baut der Browser die Seite auf: das ist Thema von VL 6.

DNS: Wie wird aus www.htwg-konstanz.de eine IP-Adresse?



Der ganze Ablauf auf einen Blick. Gleich Schritt für Schritt.

DNS ist schnell, weil es sich erinnert

```
> nslookup www.htwg-konstanz.de
Name:      cms.htwg-konstanz.de
Addresses: 2001:7c0:5f0:f020::20:31
           141.37.20.31
Aliases:   www.htwg-konstanz.de
```

Echte Abfrage vom 12.10.2026

🏷️ Record-Typen

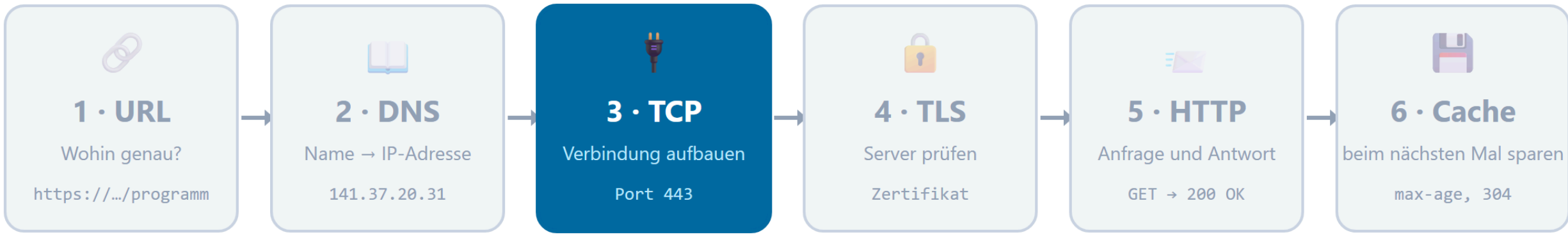
CNAME: www ist ein Alias für cms. **A:** IPv4-Adresse. **AAAA:** IPv6-Adresse. Dazu **MX** für Mail.

⏱️ TTL

Jede Antwort darf für die *Time to Live* gecacht werden. Zieht ein Server um, sehen manche noch so lange die alte IP.

Eine Anfrage, sechs Stationen

Was passiert, wenn ihr die Hackathon-Seite aufruft



Danach baut der Browser die Seite auf: das ist Thema von VL 6.

TCP: eine zuverlässige Leitung, die Zeit kostet

🔌 Was TCP leistet

Alle Daten kommen an, in der richtigen Reihenfolge. Verlorene Pakete werden neu geschickt. Dafür braucht es vorab einen **Handshake**: drei Nachrichten, bevor die erste Nutzlast fließt.

🐢 Der eigentliche Engpass

Jede Hin-und-zurück-Runde (**Round Trip**) kostet Zeit, bei fernen Servern 100 ms und mehr. Physik lässt sich nicht beschleunigen. Grigorik: Nicht die Bandbreite bremst das Web, sondern die **Latenz**.

Viele Performance-Tricks, die ihr in VL 14 kennenlernt, sparen genau solche Round Trips: Verbindungen wiederverwenden, Dateien bündeln, Server näher an die Nutzer bringen.

TLS: drei Garantien und ein Zertifikat

🙈 Vertraulichkeit

Niemand unterwegs kann mitlesen, weder das WLAN noch der Provider.

📄 Integrität

Niemand kann die Daten unbemerkt verändern.

🪪 Authentizität

Das **Zertifikat** beweist, dass der Server der ist, der er vorgibt. Unterschrieben von einer CA, der euer Browser vertraut.

Das Schloss heißt nur: Die Verbindung zu *diesem Namen* ist sicher. Auch eine Phishing-Seite unter `htwg-konstanz-login.de` kann ein gültiges Zertifikat haben.

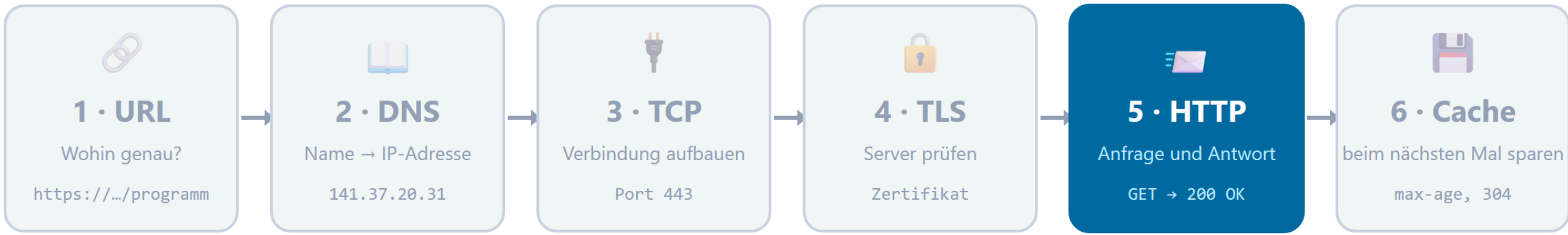
Der Browser prüft: Passt der Name im Zertifikat zum Host? Ist es noch gültig? Führt die Kette der Unterschriften zu einer bekannten CA? Erst dann fließt HTTP, verschlüsselt mit einem gemeinsam vereinbarten Schlüssel.

Halbzeit

Anwesenheit per QR-Code · Testat-Häppchen

Eine Anfrage, sechs Stationen

Was passiert, wenn ihr die Hackathon-Seite aufruft



Danach baut der Browser die Seite auf: das ist Thema von VL 6.

HTTP lesen: Anfrage und Antwort

```
> GET /03_gestylt.html HTTP/1.1
> Host: localhost:8765
> Accept: */*
>
< HTTP/1.0 200 OK
< Content-type: text/html
< Content-Length: 2093
< Last-Modified: Wed, 30 Sep 2026
```

1. `>` **Anfrage:** Methode, Pfad, Version. Dann Header, eine Leerzeile, bei Bedarf ein Body.
2. `<` **Antwort:** Statuscode in der ersten Zeile, dann Header, dann der Body, hier das HTML.
3. **Zustandslos:** Jede Anfrage steht für sich. Der Server erinnert sich an nichts, Logins kommen in VL 10.

Erzeugt mit `curl -v` gegen die Hackathon-Seite aus VL 2 auf einem lokalen Python-Server. Genau das macht ihr gleich im Hands-on.

Methoden: sicher und idempotent

📖 GET

Ressource lesen. **Sicher**: ändert nichts. **Idempotent**: zweimal wie einmal.

➕ POST

Neues anlegen oder Aktion auslösen. Weder sicher noch idempotent: Jeder Aufruf legt eine neue Anmeldung an.

✏️ PUT, PATCH, DELETE

Ersetzen, teilweise ändern, löschen. PUT und DELETE sind idempotent: Ein zweites `DELETE /events/42` löscht nichts mehr.

Ein GET darf nie etwas ändern. Ein Link `/events/42/loeschen` ist ein Fehler: Suchmaschinen, Link-Vorschauen in Chat-Apps und Browser, die vorab laden, folgen Links ungefragt.

Warum Idempotenz zählt: Browser, Proxies und Bibliotheken dürfen idempotente Anfragen bei Netzproblemen einfach wiederholen.

Statuscodes: die erste Ziffer verrät die Klasse

✅ 2xx Erfolg

`200 OK` · `201 Created` nach dem Anlegen · `204 No Content`

🔁 3xx Umleitung

`301` dauerhaft · `302` vorübergehend · `304 Not Modified` beim Caching

🤦 4xx Fehler des Clients

`400` kaputte Anfrage · `401` · `403` · `404 Not Found`

🔥 5xx Fehler des Servers

`500 Internal Server Error` · `503 Service Unavailable`

401 oder 403? `401`: Wer bist du? Es fehlt eine gültige Anmeldung. `403`: Ich weiß, wer du bist, aber du darfst das nicht. Angemeldet, aber kein Admin, heißt `403`. Das braucht ihr in M3 bei den Rollen.

Umleitungen: die HTWG im Test

```
$ curl -I http://www.htwg-konstanz.de/
HTTP/1.1 302 Found
Location: https://www.htwg-konstanz.de/
```

🔁 Was passiert

Der Browser folgt dem `Location`-Header und fragt ein zweites Mal über HTTPS. Chrome und Firefox probieren sogar von sich aus zuerst HTTPS: Im Network-Tab steht dann ein internes `307`, das nie beim Server war.

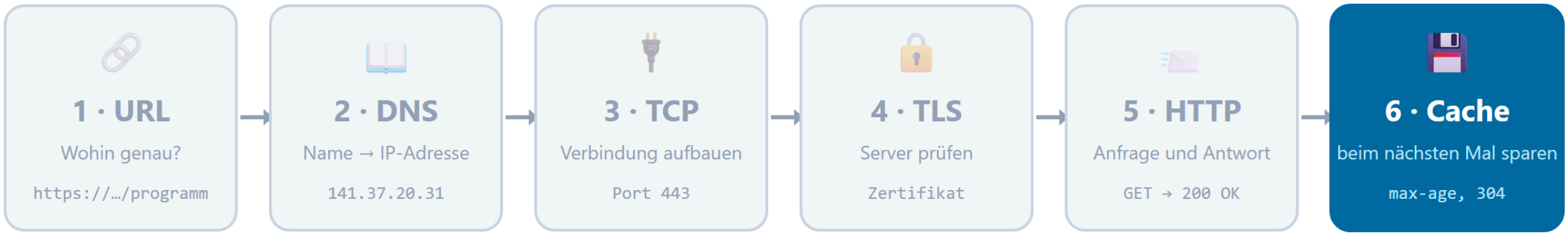
🤔 302 oder 301?

`302` heißt „vorübergehend“, der Browser fragt jedes Mal neu. Für HTTP → HTTPS wäre `301` (dauerhaft) besser, noch besser zusätzlich `Strict-Transport-Security`.

Post/Redirect/Get: Nach einem erfolgreichen `POST` antwortet der Server mit einer Umleitung auf eine Ergebnisseite. Wer dann neu lädt, wiederholt nur ein harmloses `GET`, keine doppelte Anmeldung.

Eine Anfrage, sechs Stationen

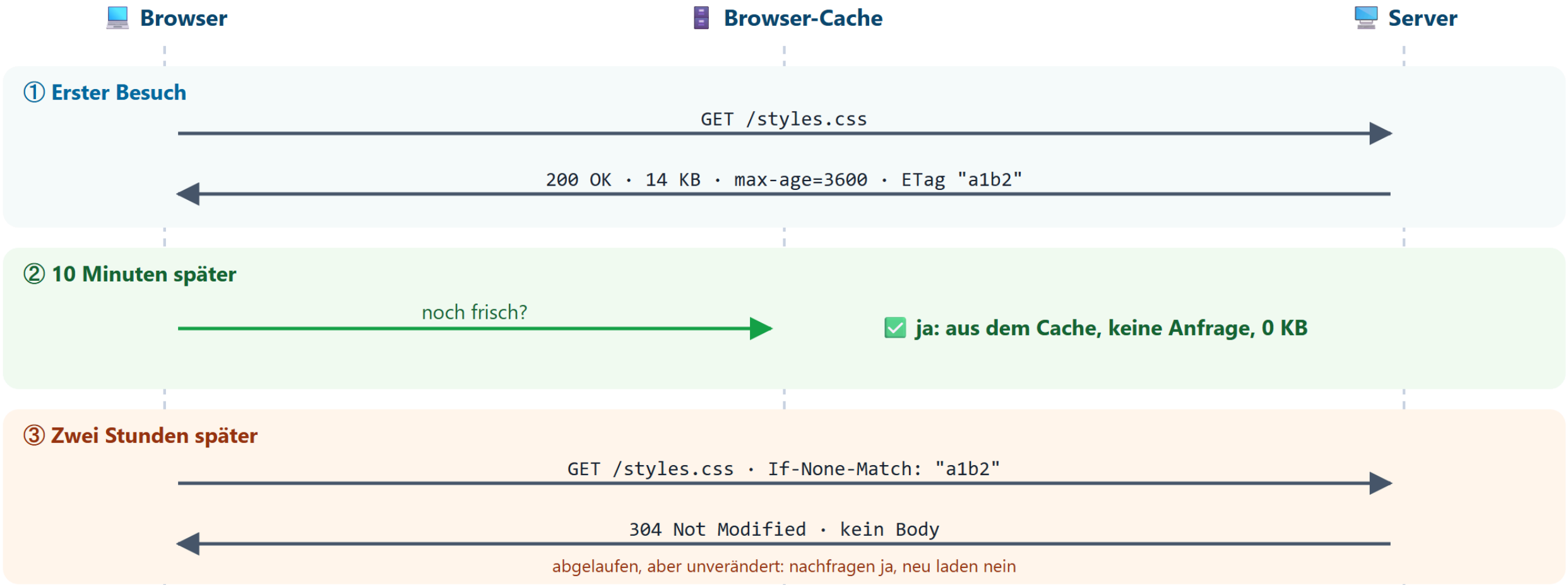
Was passiert, wenn ihr die Hackathon-Seite aufruft



Danach baut der Browser die Seite auf: das ist Thema von VL 6.

Dreimal dieselbe Datei: styles.css

Antwort des Servers: Cache-Control: max-age=3600 und ETag: "a1b2"



Lange cachen und trotzdem aktuell

```
GET /typo3temp/.../merged-7c99adcc….css
Cache-Control: max-age=31536000

GET /
Cache-Control: private, no-store
```

Echte Antworten von www.htwg-konstanz.de, 12.10.2026

Build-Werkzeuge wie Vite erzeugen solche Dateinamen automatisch. Ihr seht das in M2.

Hash im Dateinamen

Die CSS-Datei darf **ein Jahr** im Cache bleiben. Ändert sich der Inhalt, ändert sich der Name, also ist es eine neue Datei.

HTML nie lange

Die Seite selbst kommt mit `no-store`, damit Änderungen und neue Dateinamen sofort ankommen.

Ausblick und Werkzeuge

🚀 HTTP/1.1, /2, /3

/1.1: eine Anfrage nach der anderen pro Verbindung. /2: viele gleichzeitig über eine. /3: auf QUIC statt TCP, weniger Round Trips. Methoden, Statuscodes und Header bleiben gleich. Die HTWG spricht übrigens noch HTTP/1.1. Mehr in VL 14.

🧰 Eure Werkzeuge

DevTools → Network (`F12`): jede Anfrage mit Status, Größe, Zeit, Headern.
`Strg+Shift+R` lädt ohne Cache.
curl -v: HTTP ohne Browser (Windows: `curl.exe`).
nslookup: DNS prüfen.

Jetzt ihr: 30 Minuten HTTP-Detektivarbeit

Öffnet in Moodle **„VL 03 · HTTP-Detektiv“**. Zu zweit, ein Laptop mit Browser und Terminal reicht.
Eure Antworten notiert ihr direkt auf der Seite.

Übung ab 09:45: Arbeitszeit für M1

M1 ist am **Mittwoch, 21.10., 23:59** fällig. Die Übung gehört heute ganz eurem Prototyp.

🖥️ Lokal ausliefern

`npx serve` oder `python -m http.server` statt Datei per Doppelklick öffnen

🔍 Network-Tab prüfen

Alles `200`? Kein vergessenes `404` für CSS oder Bilder?

🔗 URLs planen

Sprechende Pfade, relative Links, im README kurz erklärt

Ich gehe herum und schaue mir eure Ideen und Prototypen an. Die Checkliste liegt in Moodle.

Quellen

- Ilya Grigorik: *High Performance Browser Networking*. O'Reilly, 2013, Kapitel 1 bis 4 und 9 bis 12, frei unter hpbn.co
- MDN Web Docs: *An overview of HTTP*, *HTTP caching*, *HTTP response status codes*
- RFC 9110 *HTTP Semantics*, RFC 9111 *HTTP Caching*, IETF 2022
- Messungen an www.htwg-konstanz.de mit nslookup und curl, 12.10.2026

Frameworks kommen und gehen. HTTP bleibt.

Nächste Woche: JavaScript kompakt. Sprache, Event Loop, DOM und Fetch.